

DEV2 – JAVL – Laboratoire Java**TD 14 – Modèle-Vue-Contrôleur (suite)****Résumé**

Ce TD vous propose de développer une application permettant de manipuler une base de donnée en utilisant le design pattern MVC (introduit au TD précédent).

Table des matières

1	Introduction	2
2	La base de donnée	2
3	Le modèle	3
3.1	La classe Customer	3
3.2	La classe Account	3
3.3	Tester la classe Account	4
3.4	L'énumération Status	5
3.5	La classe Transaction	5
3.6	Tester la classe Transaction	5
3.7	La classe DatabaseAccessor	5
4	La vue	6
5	Le contrôleur	6
6	La méthode principale	7

1 Introduction

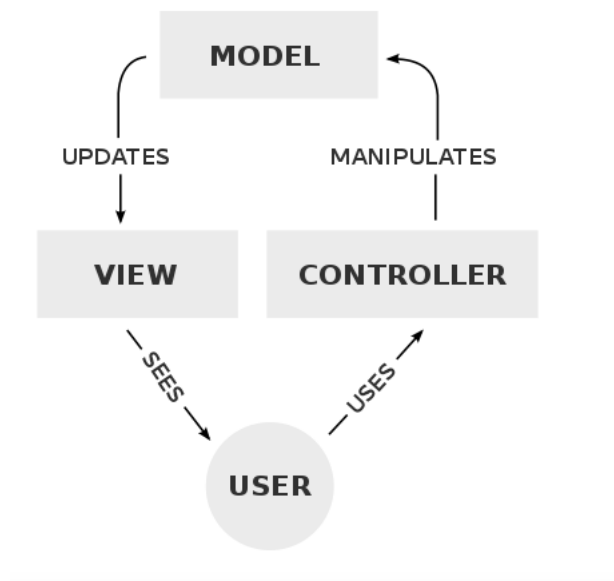
Nous allons vous guider dans l'écriture d'une application bancaire exécutant une transaction (versement d'une somme d'argent d'un compte débiteur à un compte créditeur).



L'application devra effectuer les actions suivantes :

1. Demande à l'utilisateur les numéros (au format IBAN) des comptes débiteurs et créditeurs ainsi qu'un montant à transférer du premier compte vers le second.
2. Aller chercher les informations sur ces comptes dans une base de données et afficher un message d'erreur si les comptes ne s'y trouvent pas.
3. Exécuter (si possible) la transaction et mettre la base de données à jour.

Le design pattern MVC que vous avez découvert lors de la séance précédente sera utilisé tout au long du développement :



Veuillez créer maintenant un nouveau projet Java appelé **bank** (prenez garde à bien utiliser Maven).

2 La base de donnée

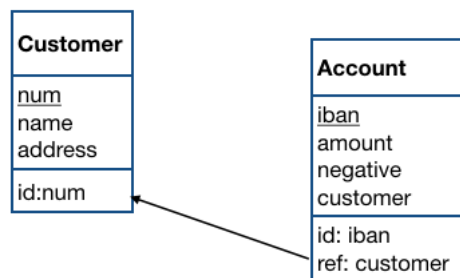
Nous utiliserons une base de donnée SQLite¹ pour l'enregistrement des données des clients et des comptes bancaires. Ajoutez dès lors les lignes suivantes dans votre `pom.xml` de façon à importer le driver nécessaire pour la connexion à la base de données :

1. Voir les slides du chapitre 1 du cours de DON2 pour plus d'information sur les systèmes de gestion de base de données et sur SQLite en particulier

```
<dependencies>
  <dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.7.2</version>
  </dependency>
</dependencies>
```

L'avantage d'une base de données SQLite est qu'elle tient en un seul fichier que nous vous fournissons : **bank.db**. Placez ce fichier dans le répertoire **src/main/resources/** de votre projet.

Le schéma de cette base de donnée vous est fourni ci-dessous (une vraie base de donnée bancaire est évidemment beaucoup plus complexe que cela, nous avons simplifié pour les besoins du TD) :



3 Le modèle

Créez maintenant un package **g12345.bank.model** dans votre projet. Les classes qui suivent seront à placer dans ce package.

3.1 La classe Customer

Créez la classe **Customer** représentant un client de la banque. Les banques encodent de nombreuses données différentes sur leurs clients mais dans le cadre de ce TD, nous nous simplifierons la vie en ne considérant que :

1. comme attributs (privés) : un numéro **num** identifiant le client au sein de la banque (un entier), un nom **name** et une adresse **address** (des chaînes de caractères),
2. un constructeur ainsi que des getters pour les différents attributs,
3. des setters pour le nom et l'adresse uniquement (nous considérerons qu'un client ne change pas de numéro).

Rappel : simplifiez-vous la vie et faites générer automatiquement les getters/setters ainsi que le constructeur par IntelliJ.

3.2 La classe Account

Celle-ci représente un compte en banque (appartenant à un client) et comportera :

1. le titulaire du compte : un attribut **owner** de type **Customer**,
2. le montant du compte, un attribut **amount** (nombre réel) ainsi qu'un attribut booléen **negative** indiquant si le compte peut être en négatif ou non,
3. le numéro identifiant le compte au format IBAN, un attribut de type **String**,

4. des getters pour tous les attributs,
5. un constructeur vérifiant que l'owner n'est pas `null` ainsi que la cohérence des attributs `amount` et `negative` (si `negative` est à `false`, le montant ne peut être négatif). Il faudrait aussi vérifier que le numéro IBAN est bien valide : nous laisserons ceci de côté pour le moment,
6. un setter pour l'attribut `negative` uniquement,
7. finalement, une méthode :

```
public void add (double amount)
```

qui ajoute la somme correspondant au solde du compte. Le montant passé en paramètre peut être négatif, auquel cas il s'agit d'un retrait. Si la somme finale est négative et que le compte ne peut aller en négatif, une exception est levée.

3.3 Tester la classe Account

Nous vous fournissons les tests de la classe `Account` ci-dessous, vérifiez qu'ils passent tous. Remarquez l'utilisation de la méthode `assertDoesNotThrow` permettant de vérifier qu'aucune exception n'est déclenchée (ce qui doit forcément être le cas si un objet est correctement instancié).

```
class AccountTest {

    Customer target_customer = new Customer(1, "TestCustomer", "TestAddress");
    String target_IBAN = "BE7810019191";

    @Test
    public void can_create_account_with_positive_amount(){
        assertDoesNotThrow(() ->
            new Account(target_customer, target_IBAN, 100, false));
    }

    @Test
    public void can_create_account_with_null_amount(){
        assertDoesNotThrow(() -> new Account(target_customer, target_IBAN, 0, false));
    }

    @Test
    public void can_create_account_with_negative_amount(){
        assertDoesNotThrow(() ->
            new Account(target_customer, target_IBAN, -100, true));
    }

    @Test
    public void cannot_create_account_with_negative_amount_if_negative_is_false(){
        assertThrows(IllegalArgumentException.class,
            () -> new Account(target_customer, target_IBAN, -100, false));
    }

    @Test
    public void cannot_go_negative_when_forbidden(){
        Account acc = new Account(target_customer, target_IBAN, 100, false);
        assertThrows(IllegalArgumentException.class,
            () -> {acc.add(-150);} );
    }

    @Test
    public void can_go_negative_when_allowed(){
        Account acc = new Account(target_customer, target_IBAN, 100, true);
        assertDoesNotThrow (() -> {acc.add(-150);});
    }
}
```

```
}
```

3.4 L'énumération Status

Une transaction bancaire peut être associée à différents status qui nous renseignent sur son état. Nous nous limiterons ici à trois status :

- ▷ **SUBMITTED** : la transaction est en cours de traitement,
- ▷ **REJECTED** : la transaction est rejetée à cause d'une erreur (solde insuffisant sur le compte débiteur, ou autre problème),
- ▷ **COMPLETED** : la transaction a été exécutée avec succès.

Créez une énumération **Status** reprenant ces trois valeurs.

3.5 La classe Transaction

Celle-ci représente une transaction bancaire entre deux comptes. Créez là et ajoutez lui :

1. deux attributs **debt** et **cred**, de type **Account**, les comptes débiteurs et créditeurs associés,
2. un attribut **amount** (un réel), montant de la transaction,
3. un attribut **status**, le statut de la transaction,
4. un constructeur recevant en paramètre et initialisant les comptes débiteurs et créditeurs ainsi que le montant. Attention, contrairement au montant d'un compte en banque, le montant d'une transaction est toujours positif (une exception doit être levée le cas échéant). Le statut de la transaction est initialisé à **SUMBITTED**,
5. des getters pour les différents attributs.

Ajoutez de plus une méthode :

```
public void execute()
```

qui se charge d'exécuter la transaction, à savoir :

1. lève une **IllegalStateException** si le statut de la transaction n'est pas à **SUMBITTED**,
2. vérifie soit que le compte débiteur peut aller en négatif, soit que le montant disponible sur celui-ci est supérieur au montant de la transaction. Si ce n'est pas le cas, le statut de la transaction passe à **REJECTED** et aucun des comptes n'est mis à jour,
3. sinon : le montant est soustrait du compte débiteur, ajouté au compte créditeur, et le statut passe à **COMPLETED**.

3.6 Tester la classe Transaction

Ecrivez des tests unitaires complets pour la classe **Transaction**.

3.7 La classe DatabaseAccessor

Notre modèle est presque complet, mais il nous reste à implémenter une classe permettant de lire et mettre à jour la base de données. Comme cela fait intervenir des notions non abordées en Dev2, nous vous la fournissons : ajoutez-là simplement à votre package **model**.

Lisez la Javadoc des différentes méthodes de la classe pour comprendre comment elles fonctionnent : vous allez devoir utiliser cette classe dans le contrôleur !

Bases de données et paradigme orienté objet

Comme vous pouvez le constater en parcourant le code de la classe, faire correspondre les données des objets du programme avec celles des tables de la base de données relationnelle demande un certain nombre de lignes de code. Des outils ont été inventés de façon à réduire les coûts de productivité liés à ces différences de paradigmes, nous en parlerons dans d'autres cours (voir notamment le dernier chapitre du cours de DON2 sur les modèles alternatifs de données).

4 La vue

Créez un package `g12345.bank.view` contenant une classe `BankingView`, et qui contiendra des méthodes fort simples :

1. `void displayTitle()` qui affiche un titre et un message de bienvenue à l'utilisateur,
2. `String askIBANDeb()` et `String askIBANCred()` qui demandent à l'utilisateur de rentrer les comptes IBAN des débiteurs et créditeurs et les renvoient,
3. `int askAmount()` qui demande le montant de la transaction et le renvoie,
4. `void displayTransaction(Transaction t)` qui affiche un message informant l'utilisateur du statut de la transaction. Pensez à bien distinguer les différents statuts (pour des raisons évidentes de clarté auprès de l'utilisateur).

Vous êtes libre d'ajouter toute autre méthode jugée utile pour améliorer l'affichage, mais **tout l'affichage doit se faire dans cette classe** (et surtout pas dans le modèle ni le contrôleur) !

5 Le contrôleur

La classe `Controler` finalement, à placer dans un package `g12345.bank.controler`, aura l'aspect suivant :

```
package sre.bank.controler;

import sre.bank.view.BankingView;
import sre.bank.model.DatabaseAccessor;

public class Controler {
    private BankingView view;

    private DatabaseAccessor dbAccessor;

    public Controler(BankingView view, String dbFile){
        this.view = view;
        this.dbAccessor = new DatabaseAccessor(dbFile);
    }

    public void startPayment(){
        // Complétez le code ici ...
    }
}
```

Complétez le code de la méthode `startPayment` de façon à ce qu'elle :

1. affiche le titre et demande les numéros de compte IBAN des débiteurs et créditeurs ainsi que le montant de la transaction (via les méthodes de la `view` uniquement !),
2. lise les données des comptes débiteurs et créditeurs dans la base de donnée via l'objet `dbAcc` (allez relire la javadoc des méthodes de la classe si nécessaire),
3. affiche un message d'erreur à l'utilisateur si données des comptes sont introuvables,
4. sinon, crée un nouvel objet `Transaction` et tente de l'exécuter. Affiche le résultat au moyen de la méthode `displayTransaction` de la vue,
5. mette à jour la base de données au moyen de l'objet `dbAcc` si la transaction a correctement été effectuée.

6 La méthode principale

Finalement, la méthode principale (à mettre dans une classe `BankingApp`) se limite à ceci :

```
import sre.bank.controler.Controler;
import sre.bank.view.BankingView;

public class BankingApp {

    public static final String DB_FILE = "src/main/resources/bank.db";

    public static void main(String[] args) {
        Controler c = new Controler(new BankingView(), DB_FILE);
        c.startPayment();
    }
}
```

Exécutez là et tentez de faire un payment pour vérifier que tout fonctionne correctement.